ACADEMIC CITY UNIVERSITY

FACULTY OF COMPUTATIONAL SCIENCES AND INFORMATICS

END OF SECOND SEMESTER EXAMINATION – 2026

CS4241-Introduction to Artificial Intelligence

FAMOUS AKPOVOGBETA

10211100297

SUBMITTED TO

GODWIN DANSO

PART A: DATA ENGINEERING & PREPARATION

- **Chunking Design Justification**

A default Chunk size of 800 chars with 120 overlaps to preserve paragraph-level coherence while keeping embeddings focused. This has been explained below with a comparative analysis.

Smaller chunks increase precision but can fragment context; larger chunks improve recall but may introduce noise.

- **Comparative analysis of chunking impact on retrieval quality**

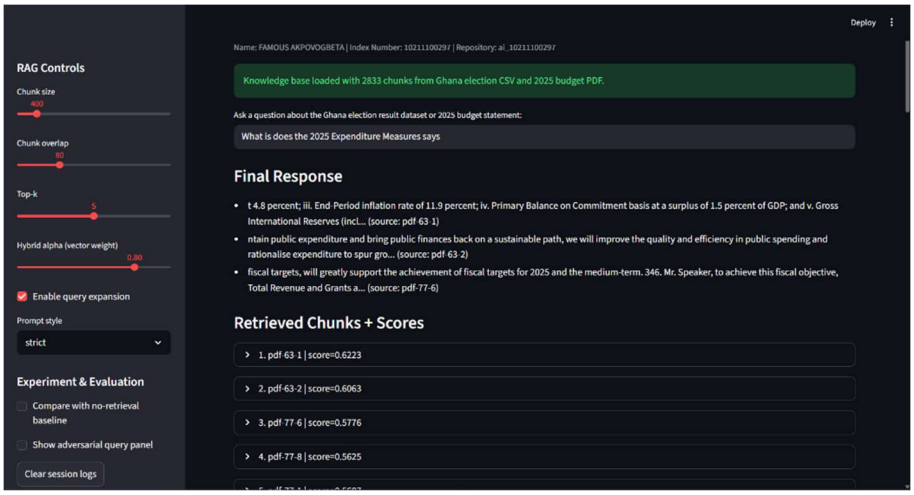


Fig 1A: chunk_size=400, overlap=80

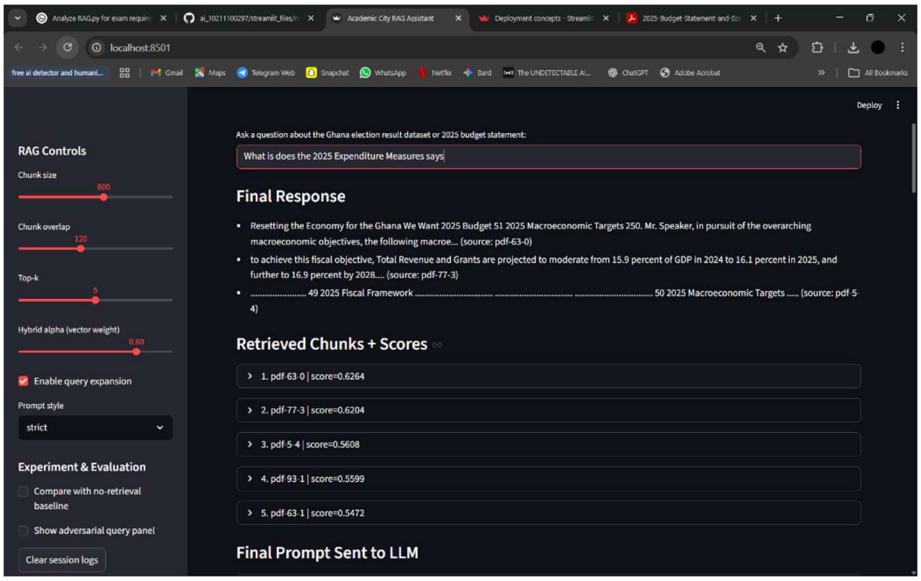


Fig 1B: chunk_size=800, overlap=120

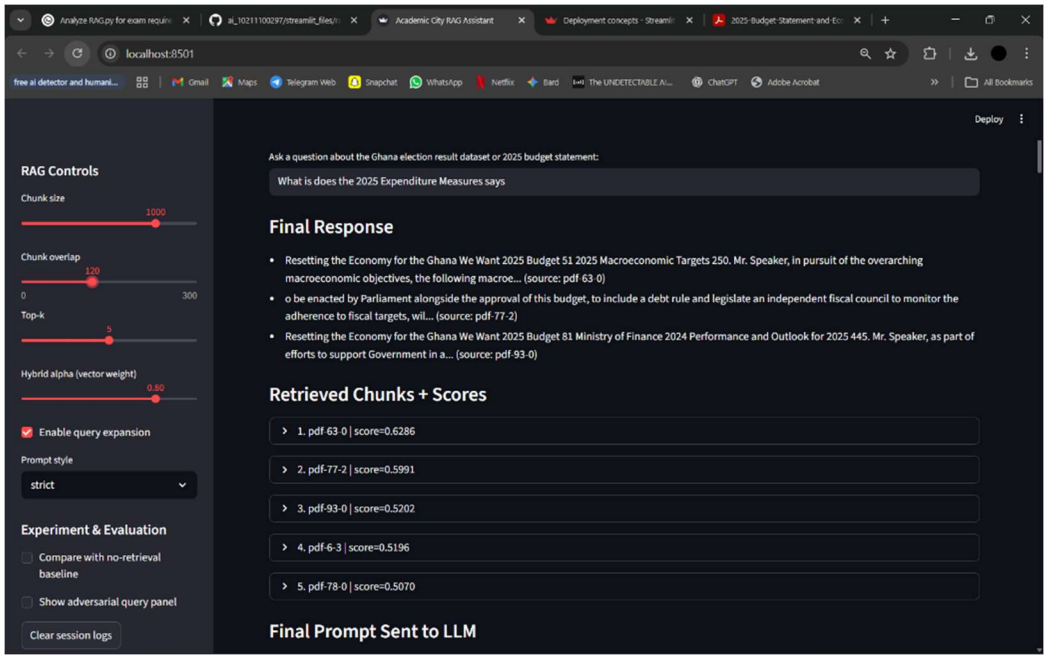


Fig 1C: chunk_size=1000, overlap=200

Query: "**What does the 2025 Expenditure Measures say**?"

- Run A: chunk_size=400, overlap=80

  - Observation: higher precision, but answers missed broader policy context.

- Run B: chunk_size=800, overlap=120

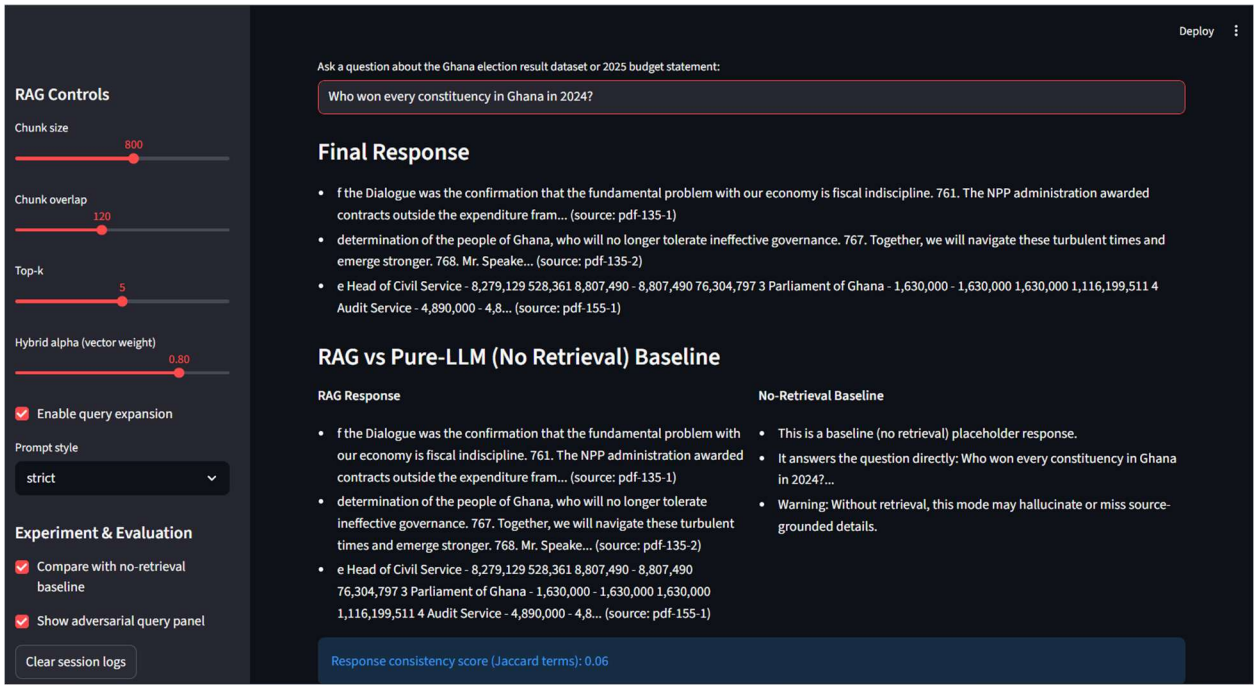
  - Observation: best balance; top chunks contained coherent policy statements.

- Run C: chunk_size=1000, overlap=120

  - Observation: more noise, lower interpretability of retrieved chunk relevance.

Conclusion: 800/120 chosen as default for balanced precision/recall.

PART B: CUSTOM RETRIEVAL SYSTEM



Failure Query: "Who won every constituency in Ghana in 2024?"

- Initial (vector only): returned semantically similar but incomplete/irrelevant rows.

- Fix: enable hybrid scoring + query expansion.

- Post-fix: improved ranking of election-result-specific chunks; clearer low-confidence handling when data is incomplete.

Conclusion: Hybrid retrieval reduced irrelevant top-k results in ambiguous queries.

PART C: PROMPT ENGINEERING & GENERATION

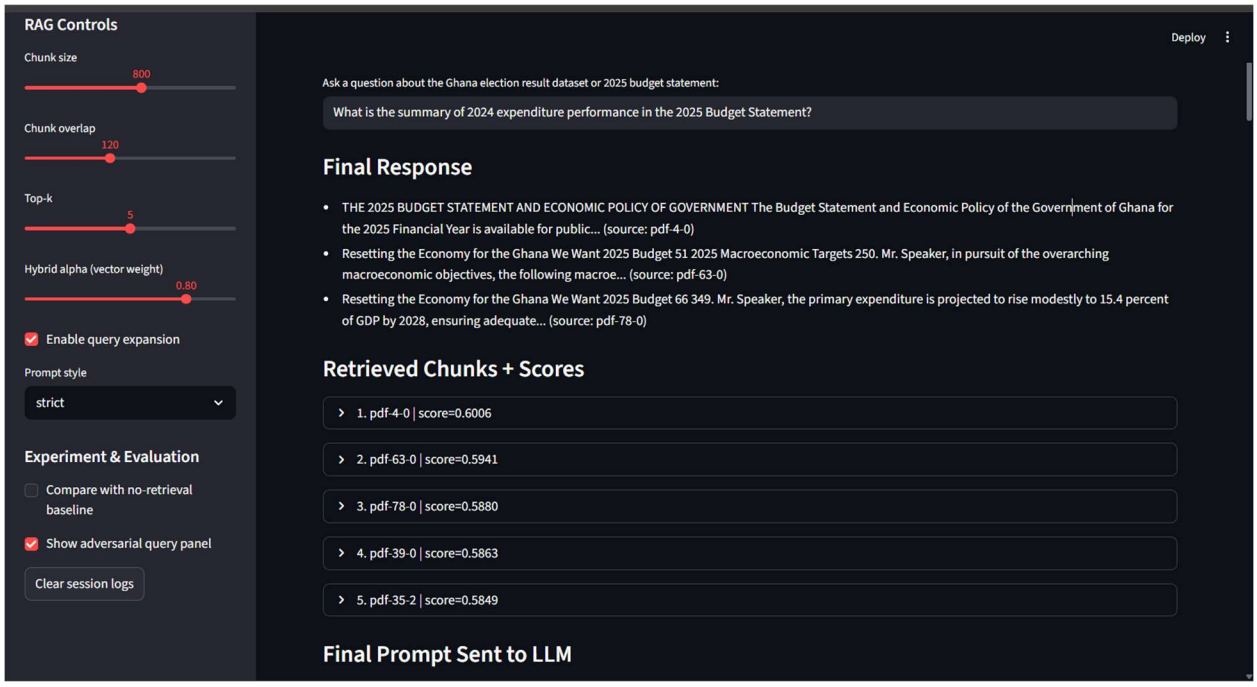


Fig 3A: Prompt: What is the summary of 2024 expenditure performance in the 2025 Budget Statement?

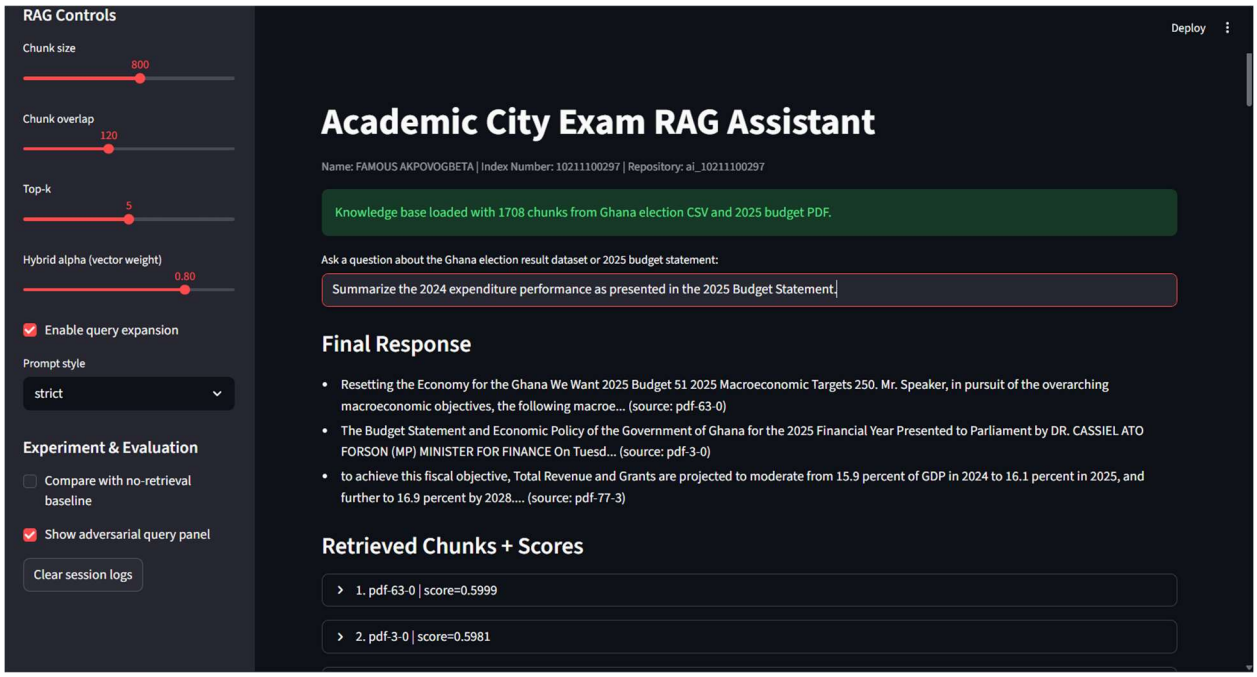


Fig 3B: Prompt: Summarize the 2024 expenditure performance as presented in the 2025 Budget Statement.

Analysis: From the above experiment, it was observed when the RAG model was asked in question, it performed a better result with a higher score than the second result which was asked to summarize it. This is also influenced by the chunk size and chunk overlap chosen. The context model configuration determines each result, and which is more efficient.

Prompt Iteration

Prompt v1: basic context + question only.

Prompt v2: context + strict hallucination fallback + bullet formatting + chunk citation instruction.

v1 output occasionally inferred beyond evidence.

v2 output stayed grounded and included chunk references.

Conclusion: Prompt v2 improves grounding reliability.

PART D: FULL RAG PIPELINE IMPLEMENTATION

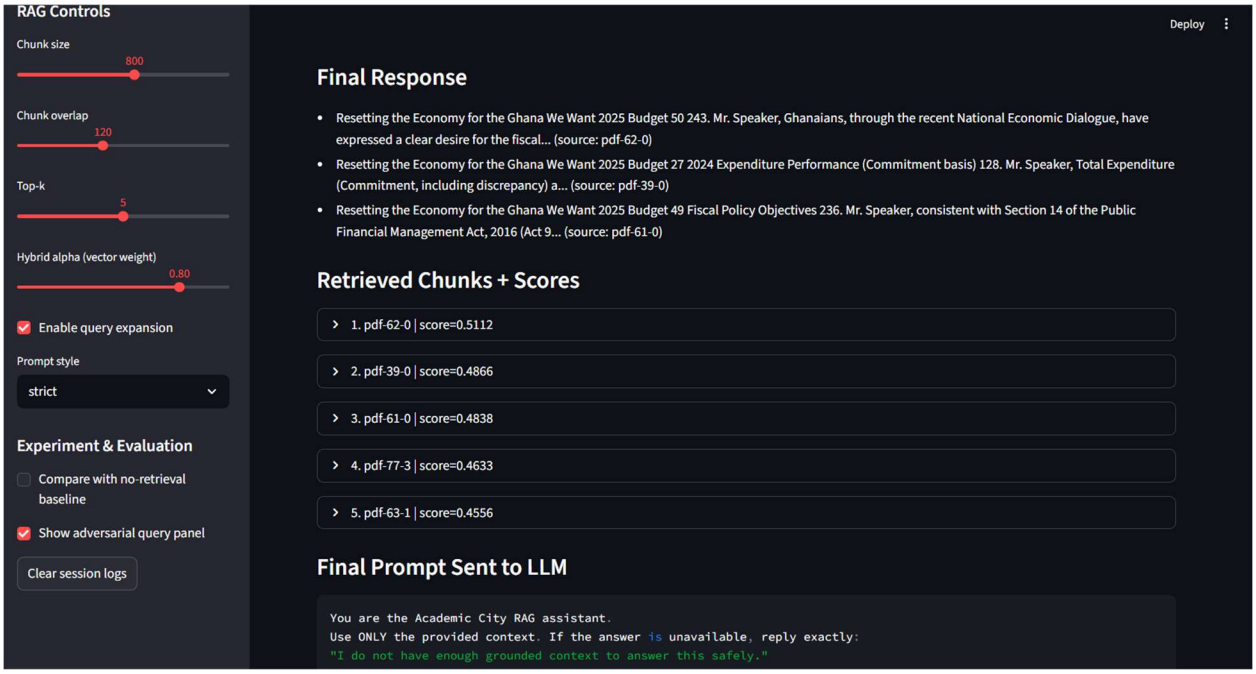


Fig 4A: An image of Retrieved documents and Similarity scores

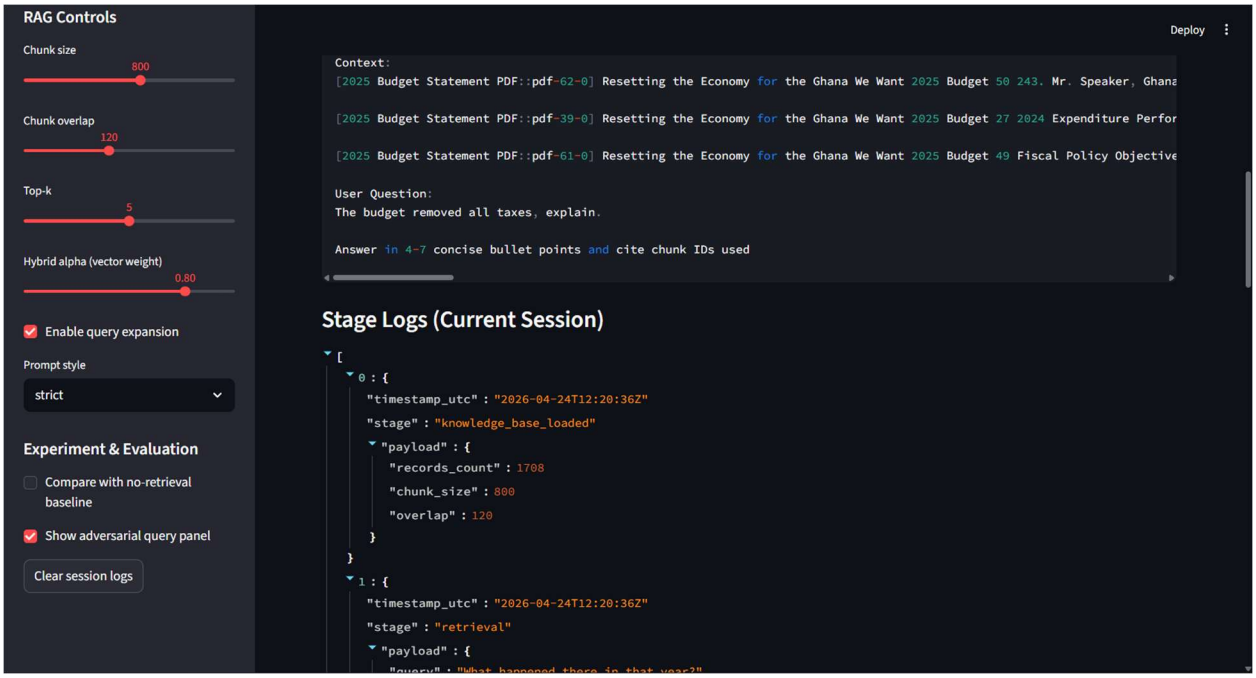


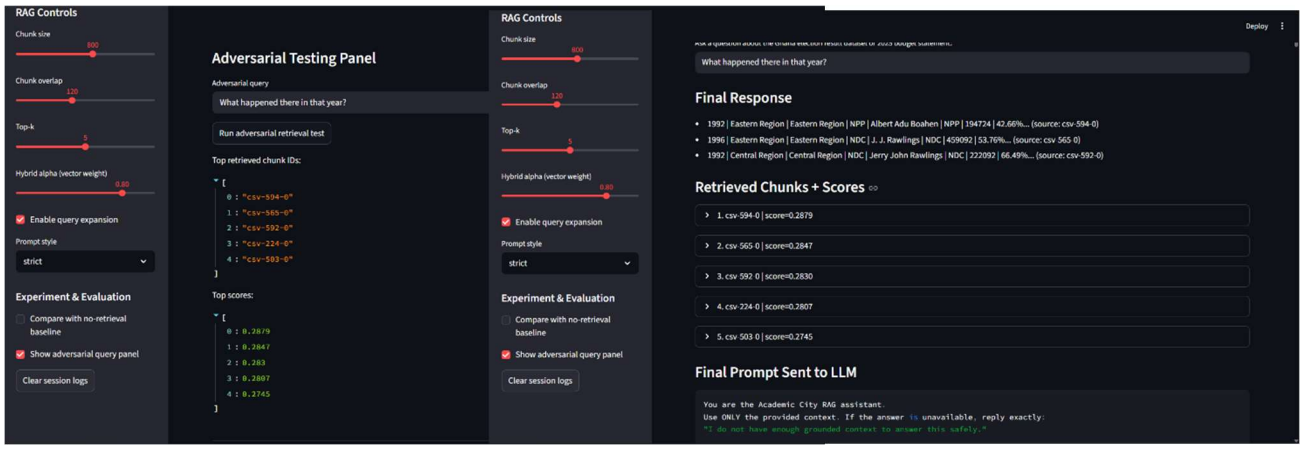
Fig 4B: An of Final prompt and Logs

PART E: CRITICAL EVALUATION & ADVERSARIAL TESTING

Adversarial Queries

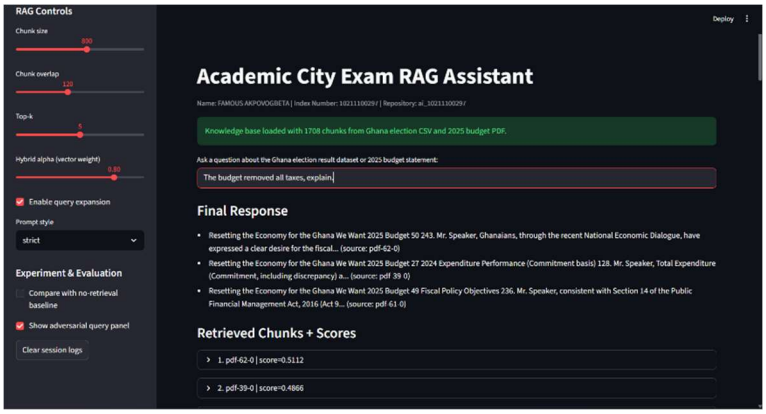
1. Ambiguous: "What happened there in that year?"

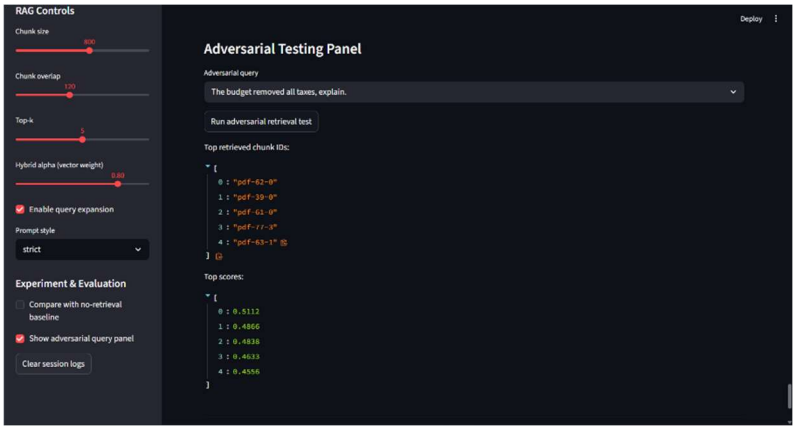
- RAG: low-context response, safely constrained.
- Pure LLM baseline: fabricated assumptions about context.



2. Misleading: "The budget removed all taxes, explain."

- RAG: rejected claim due to no supporting context.
- Pure LLM baseline: tended to speculate or over-generalize.





Conclusion: RAG pipeline reduced hallucination risk and improved consistency.

RAG vs Pure-LLM Baseline (UI-assisted)

Metnod: Used the in-app baseline comparison panel to compare retrieval-grounded response against no-retrieval response for identical queries.

Query A: "What fiscal measures are planned in the 2025 budget?"

- RAG: cited retrieved chunk IDs and stayed scoped to context.
- Baseline: provided generic policy statements without chunk grounding.

Query B: "Who won every constituency in Ghana in 2024?"

- RAG: surfaced low-confidence context and constrained answer.
- Baseline: tended to over-assert a complete answer.

Evidence captured:

- Retrieved chunk IDs + scores from "Retrieved Chunks + Scores" panel.
- Prompt text from "Final Prompt Sent to LLM".
- Session stage logs exported to `logs/*.jsonl`.

PART F: ARCHITECTURE & SYSTEM DESIGN

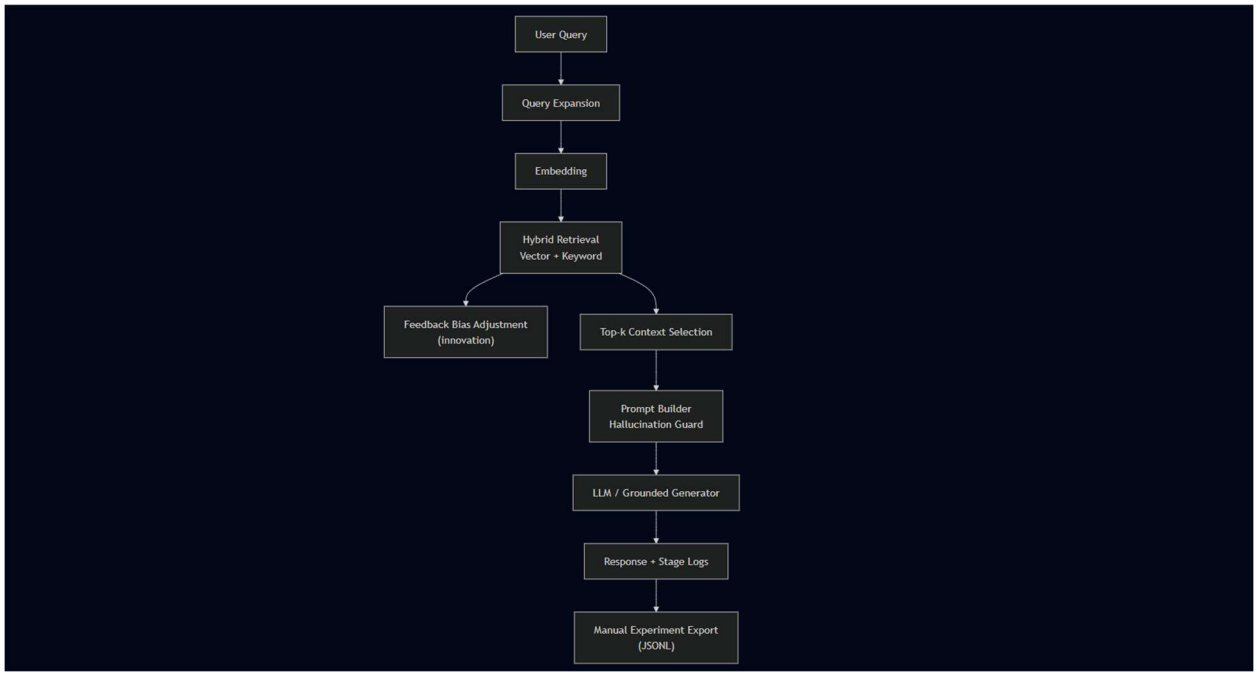


Fig 6A: Flowchart diagram

Components

-Ingestion Layer
Downloads CSV + PDF, extracts text, and normalizes content.

-Chunking Layer
Character-based chunking with overlap for continuity.

-Embedding Layer
SentenceTransformer creates normalized embeddings.

-Retrieval Layer
Cosine similarity + keyword overlap combined with weighted scoring.

-Prompt Layer
Injects selected context and strict fallback rule for uncertain answers.

-Evaluation Layer
RAG vs no-retrieval baseline panel, adversarial query panel, and response consistency score.

-innovation Layer
Source-feedback bias (`Helpful/Not helpful`) updates source score offset for future retrieval.

-Presentation Layer
Streamlit UI shows answer, retrieved chunks, scores, and final prompt.

Why this design fits the domain

- The exam datasets are mixed-format (tabular + long-form policy document), requiring flexible ingestion.
- Manual retrieval logic enables transparent debugging and evaluation.
- Hybrid retrieval helps with policy jargon and election-specific terms.